Arabic Bank Check Amount Extraction and Processing

Course Project Report
ICS 472: Natural Language Processing
Department of Computer Science
student.name@example.com

Abstract

This report describes an end-to-end system for Arabic bank check amount processing. The system localizes the legal and courtesy amount fields, recognizes the courtesy amount as a digit sequence, recognizes the legal amount as an Arabic handwritten text sequence, converts the legal text into a numeric value using rule-based postprocessing, and verifies consistency between the two amount fields. The current draft focuses on the problem definition, dataset organization, methodology, and experiment setup. Quantitative results are intentionally left to be filled after running validation experiments and, only after selecting the final configuration, the final test-set evaluation cells in the project notebook.

1 Problem definition

Arabic bank checks contain two complementary amount fields: the courtesy amount, written numerically, and the legal amount, written in words. A reliable check-processing system must first extract both fields from the full check image, then recognize each amount, then verify that the two decoded values agree. This project follows the four-part structure defined in the course specification:

- **Part A:** extract the legal and courtesy amount regions from a full check image.
- **Part B:** recognize the courtesy amount as a sequence of digits.
- **Part C:** recognize the legal amount as a sequence of characters or legal tokens.
- **Part D:** convert the legal amount into digits and verify it against the courtesy amount.

The task is challenging because bank checks contain stamps, background texture, variable handwriting, skew, noise, and inconsistent spacing. The legal amount is especially difficult because Arabic handwriting is cursive and because visually small errors may change a numeric word. The final objective is not only isolated recognition accuracy, but also end-to-end verification of whether the legal and courtesy amount fields represent the same value.

2 Dataset

2.1 Provided folders

The project uses the local dataset folders in the repository root. The active training inputs are:

- `Images/`: full check images in TIF format.
- `BoundingBoxes/`: YOLO-format annotations for legal and courtesy amount regions.
- `CourtesyAmounts/`: tokenized courtesy amount labels.

Preprint.

- `LegalAmounts/`: tokenized legal amount labels used by the legal recognizer.

Raw/reference folders such as `CourtesyAmounts_raw/` and `LegalAmounts_raw_text/` are kept for traceability, but they are not active notebook inputs. The dataset is licensed and should not be shared publicly.

2.2 Annotation structure

The bounding-box files use normalized YOLO rows with the format

$$\texttt{class\_id} \quad x_c \quad y_c \quad w \quad h.$$

Class 0 is treated as the legal amount and class 1 is treated as the courtesy amount. Each valid check is expected to have one legal box and one courtesy box. The output order required by the project is the check filename, followed by the courtesy bounding box, then the legal bounding box.

The courtesy labels are parsed into digit sequences. Separator symbols and dot-like marks that do not represent target digits are ignored during label conversion. The legal labels are active subword/legal-token annotations. For CTC training, the notebook joins these legal tokens into a character sequence, while still retaining the original token list for a WER-style token error rate.

2.3 Training, validation, and test protocol

The notebook uses a fixed 85% training and 15% validation split before the final test set is touched. With 1800 check images this corresponds to 1530 training samples and 270 validation samples. The file `ac00048` is intentionally treated as a special case without active annotations, so the executable split uses 1799 valid labeled samples: 1529 for training and 270 for validation.

The final instructor-provided test data are kept separate under `CheckImages-Test/` and `CheckAnnotations-Test/`. The test cells are locked by default and should be run only after validation artifacts for Parts A–D have been produced, error analysis and ablation studies have been reviewed, and the final hyperparameter configuration has been selected.

3 Methodology

3.1 End-to-end design

The system is organized as a four-stage pipeline that follows the structure of the course project and the Arabic check-processing reference report. The first stage detects the two amount regions in the full check image. The second stage recognizes the courtesy amount as a digit sequence. The third stage recognizes the legal amount as Arabic handwriting. The fourth stage converts the recognized legal text into a numeric amount and verifies it against the courtesy amount.

This decomposition is important because the project is not a single image-classification problem. Part A is a visual localization task, while Parts B and C are sequence-recognition tasks where the output length is not known in advance. The NLP contribution appears in the target representation, CTC sequence modeling, edit-distance evaluation, Arabic text normalization, fuzzy matching of legal amount tokens, and rule-based semantic conversion from Arabic amount words into digits. The final verification stage then combines the two recognized sequences and tests whether they express the same amount.

3.2 Data validation and shared split

Before any model is trained, the notebook performs a dataset audit. The active training folders are `Images/`, `BoundingBoxes/`, `CourtesyAmounts/`, and `LegalAmounts/`. Every usable image identifier must have one full check image, one bounding-box file, one courtesy label, and one legal label. Bounding boxes must be normalized YOLO rows with five fields, valid class IDs, and coordinates inside the image domain. The class convention is fixed as class 0 for the legal amount and class 1 for the courtesy amount.

The dataset contains 1800 check images. The file `ac00048` is intentionally treated as a special case without active annotations, so the executable supervised set contains 1799 samples. A single

deterministic 85/15 split is written to `results/partA_split.json`. The same split is reused by Parts A–D. This prevents a common pipeline error where the detector, recognizers, and verifier are evaluated on different validation examples.

3.3 Part A: legal and courtesy amount extraction

Part A is implemented as two-class object detection with YOLOv8. The detector receives the full check image and predicts one box for the legal amount region and one box for the courtesy amount region. The YOLO dataset is generated from the audited annotations by copying the corresponding images and label files into train and validation folders. The model is fine-tuned from `yolov8n.pt`, which is appropriate for a course project because it gives a compact detector while still using a modern object-detection backbone.

The training configuration uses 100 epochs, batch size 16, image size 640, AdamW optimization, initial learning rate 0.001, momentum 0.9, weight decay 0.0005, and automatic mixed precision when available. The best and last trained weights are copied to stable checkpoint paths under `checkpoints/partA_yolov8n/`. The notebook also saves the dataset YAML file, split metadata, prediction CSV files, and visual error examples.

At inference time, detections are postprocessed class by class. For each check image, the highest-confidence prediction is selected for class 0 and class 1. If a class is missing, a zero box is emitted instead of silently dropping the sample. This makes missed detections visible in the output file and ensures that they are counted as failures during IoU evaluation. The required submission order is preserved as check filename, courtesy amount box, and legal amount box.

3.4 Part B: courtesy amount recognition

Part B uses the trained Part A detector to crop courtesy amount patches from the full check images. The target sequence is read from `CourtesyAmounts/`. Non-target separators, dot markers, and the blank token used by the annotation convention are removed so that the final target contains only digits. This matches the project requirement that the output be the amount as a sequence of digits without special characters.

The courtesy recognizer is a CNN-BiLSTM-CTC model. The convolutional layers convert the grayscale crop into a sequence of local visual features. The bidirectional LSTM layers model context from both directions along the writing line. The CTC loss is used because the dataset provides only whole-string labels, not character-level or digit-level segmentation. During decoding, CTC blanks are removed and repeated predictions are collapsed to produce the final digit string.

The preprocessing is intentionally conservative. Courtesy amount zeros and separators are often small dot-like marks, so aggressive denoising could remove valid signal. Each crop is converted to grayscale, resized to the fixed courtesy recognizer size, converted to a tensor, and normalized to $[-1, 1]$. The training setup uses RMSprop with learning rate 10^{-4}, gradient clipping, up to 500 epochs, and early stopping with patience 20 based on validation loss. The best checkpoint, vocabulary, validation predictions, edit-distance table, and confusion matrix are saved under the project output directories.

3.5 Part C: legal amount recognition

Part C recognizes the handwritten legal amount using a deeper CNN-BiLSTM-CTC recognizer. The input patch is the legal amount crop produced by the detector. This follows the reference methodology: convolutional layers learn visual features, recurrent layers model the sequential Arabic handwriting, and CTC trains the recognizer without requiring manually segmented characters.

The legal recognizer uses a 15-layer convolutional feature extractor, adaptive pooling to a fixed sequence length of 84, two bidirectional LSTM layers, and a linear classifier over the legal character vocabulary plus the CTC blank. Legal crops are resized to 140×1340, converted to grayscale tensors, normalized to $[-1, 1]$, and horizontally mirrored so the Arabic right-to-left writing direction is aligned with the model's sequence traversal. Optimization uses Adam with learning rate 10^{-4}, step decay by a factor of 0.9 every 10,000 optimizer steps, gradient clipping, up to 500 epochs, and early stopping with patience 20.

The active `LegalAmounts/` labels are legal-token annotations. For CTC learning, these tokens are joined into a character sequence. The original token sequence is kept for token-level analysis and conversion. This distinction is important: the reported character error rate is a true character-level edit metric, while the word-style metric in the notebook is best interpreted as a legal-token error rate unless fully word-boundary labels are introduced. The notebook saves validation predictions, character-level error analysis, token-level error analysis, and a confusion matrix that includes substitutions, insertions, and deletions.

3.6 Part D: legal conversion and verification

Part D converts the predicted legal text into a numeric amount and compares it with the predicted courtesy amount. This stage is rule-based because legal check amounts belong to a constrained numeric language. The converter normalizes Arabic character variants, removes nonnumeric filler where appropriate, segments compact text, maps number words to values, applies scale words such as hundred and thousand, and handles common currency/final-form words.

The converter also uses minimum edit distance to handle recognition errors. If a predicted token is not directly found in the legal dictionary, candidate dictionary tokens are ranked by Levenshtein distance. When multiple numeric conversions are plausible, the recognized courtesy amount is used as contextual evidence to select the legal candidate that agrees with the numeric field. This implements the mutual-improvement idea in the reference report: the legal and courtesy fields are not treated as isolated recognizers at the final decision stage.

The final verifier reports agreement between the converted legal amount and the enhanced courtesy amount. It also reports legal conversion accuracy against available ground truth, not only legal-courtesy agreement, because two wrong predictions can sometimes agree with each other. A targeted zero-correction step is included for one insertion or deletion of zero in the courtesy amount when the legal conversion provides a consistent value. This correction is motivated by Arabic/Indian handwritten zeros, which can be visually small and easily deleted or inserted by the recognizer.

3.7 Artifacts and reproducibility

The notebook is designed to be rerun locally or in Google Colab with the project stored in Google Drive. Path variables are derived from the project root and all generated artifacts are written under project-controlled directories. Model checkpoints are saved under `checkpoints/`; metrics, predictions, plots, split files, and error-analysis examples are saved under `results/`. The final test section is locked by two explicit notebook flags, `HYPERPARAMETER_DEVELOPMENT_COMPLETE` and `RUN_FINAL_TEST_EVALUATION`. These safeguards reduce accidental test-set leakage and make the experiment history easier to inspect.

4 Experiments and results

4.1 Experiment setup

The experimental protocol uses validation-driven development. The active supervised set contains 1799 examples after excluding the special case `ac00048`. The fixed 85/15 split gives 1529 training examples and 270 validation examples. The validation set is used for model selection, hyperparameter comparison, ablation studies, and error analysis. The instructor-provided test set under `CheckImages-Test/` and `CheckAnnotations-Test/` is reserved for the final evaluation only.

The development workflow is:

1. audit all active annotations and stop if any required training label is missing or malformed;
2. train Part A on the 85% training split and select the detector checkpoint using validation localization metrics;
3. crop courtesy and legal patches for the shared train and validation identifiers;
4. train Parts B and C on their training crops and select checkpoints by validation loss and sequence metrics;

5. evaluate Parts B and C with edit-distance metrics, confusion matrices, and incorrect-sample tables;
6. evaluate Part D on validation predictions and inspect legal conversion failures, mismatches, and false positives;
7. compare planned ablations on the validation set, including Part C with and without data augmentation, and choose one final hyperparameter configuration;
8. unlock the final test section and run it once using the selected checkpoints.

This protocol is intentionally conservative. The final test set should not be used to choose training length, model size, preprocessing, augmentation, decoding rules, or postprocessing thresholds. If a change is made after looking at validation errors, the notebook should be rerun on the validation split before the final test flags are enabled.

4.2 Model configuration summary

Table 1: Primary model configuration used by the project notebook. Final values should be reported after the validation run is completed.

Part	Model	Main settings
A	YOLOv8 detector	100 epochs, batch 16, image size 640, AdamW, learning rate 0.001
B	CNN-BiLSTM-CTC	RMSprop, learning rate 10^{-4}, early stopping patience 20
C	CNN-BiLSTM-CTC	Adam, learning rate 10^{-4}, 15 convolution layers, 84 time steps; augmented run saved as a separate ablation checkpoint
D	Rule-based verifier	Arabic normalization, edit distance, courtesy-guided conversion

4.3 Evaluation metrics

Part A is evaluated using mean IoU and accuracy at IoU thresholds 0.50, 0.75, and 0.90. Accuracy at threshold t is the percentage of samples whose predicted box has IoU at least t with the ground-truth box. The metrics are reported separately for legal boxes, courtesy boxes, and the combined set of all boxes. Missing predictions are counted as zero-IoU cases.

Part B is evaluated using digit-level edit accuracy:

$$\text{Accuracy} = \left(1 - \frac{I + D + S}{N}\right) \times 100,$$

where I, D, and S are insertions, deletions, and substitutions, and N is the number of ground-truth digits. The notebook also reports the percentage of amounts with no errors, exactly one error, and two or more errors. These summary buckets match the course specification and make the sequence errors easier to interpret than a single average.

Part C is evaluated using character error rate and legal-token error rate:

$$\text{Error Rate} = \frac{I + D + S}{N} \times 100.$$

The character metric uses individual Arabic characters. The token metric uses the active legal-token labels and should be reported as a legal-token error rate unless explicit word-level labels are added. The notebook also saves a character confusion matrix with insertion and deletion categories because these errors are common in CTC decoding and are not visible in a substitution-only matrix.

Part D is evaluated using legal conversion accuracy, legal-courtesy matching accuracy, empty-conversion count, number of courtesy zero corrections, number of courtesy-guided legal candidate selections, and false positives against available ground truth. Legal conversion accuracy measures whether the legal text was converted into the correct numeric value. Legal-courtesy matching accuracy measures whether the two predicted fields agree. Both are necessary because internal agreement alone does not prove that the amount is correct.

4.4 Ablation and model-selection plan

The notebook contains hooks for validation-only ablation studies. The most important comparisons are legal recognition with and without augmentation, recognition from ground-truth crops versus YOLO-extracted crops, direct legal conversion versus courtesy-guided conversion, and verification before versus after zero correction. The augmentation ablation uses the same 85/15 identifiers and trains a separate Part C model with zoom-in, small left and right rotations, and optional elastic distortion. This allows the report to compare CER, legal-token error rate, and exact sequence accuracy against the plain Part C baseline without overwriting the primary checkpoint. These ablations are aligned with the reference report because they separate modeling errors, crop-quality errors, tokenization errors, and postprocessing errors.

Some ablations require full retraining. For example, comparing subword, enhanced subword, word-level, enhanced word-level, and character-level legal tokenization requires separate label construction and separate CTC training runs. The augmentation ablation is implemented as a controlled retraining run and should be reported only after both baseline and augmented validation artifacts are regenerated and saved.

4.5 Saved outputs

All experiment outputs are saved under `results/`. The notebook is expected to save JSON metrics, CSV prediction tables, training curves, confusion matrices, incorrect-example tables, visual detection examples, validation summaries, and final test summaries. These files are the source of the final numerical tables in the report. Until those artifacts are regenerated with the 85/15 split, this report intentionally describes the setup and leaves the final numeric results to be filled from the saved outputs.

5 Discussion including error analysis

The error analysis is organized by pipeline stage. For Part A, the notebook records the weakest IoU cases and saves visual examples with ground-truth boxes in green and predictions in red. Expected detection errors include small shifts under strict IoU 0.90, over-extended legal crops, missed compact courtesy regions, and interference from stamps or background text.

For Part B, the key errors are insertions and deletions in the digit sequence, especially around zero-like marks and separators. Therefore, the confusion matrix includes explicit insertion and deletion rows rather than only substitutions. The error table shows true and predicted digit strings, edit counts, and edit operations.

For Part C, the main errors are character substitutions among visually similar Arabic shapes, missed characters in cursive handwriting, and unstable spacing or token boundaries. Some legal-token mistakes involve nonnumeric terms, such as currency or final phrase tokens, and may not affect the converted value. Numeric-token substitutions are more serious because they can change the final amount.

For Part D, errors are separated into empty legal conversions, legal-courtesy mismatches, and false positive verifications against available ground truth. A false positive is defined as a case where the predicted legal and courtesy values match each other but do not match the true amount. This distinction is important because the final verifier can appear internally consistent while still being wrong.

6 Conclusions including the limitations

This project implements an end-to-end Arabic check amount processing pipeline aligned with the course requirements and the reference methodology. The design combines YOLOv8 field extraction, CNN-BiLSTM-CTC recognition for both courtesy and legal fields, rule-based legal-to-digit conversion, and mutual verification between the two amount fields.

The main limitations are expected from the dataset and pipeline structure. The detector is sensitive to annotation tightness, especially for long legal regions. The courtesy recognizer may confuse dot-like

zeros or separators. The legal recognizer depends on the available tokenization scheme and currently reports a legal-token error rate rather than a true word-boundary WER. The rule-based converter depends on dictionary coverage and may fail on unusual spellings, extra nonnumeric phrases, or unseen legal amount forms. Finally, the augmentation ablation requires an extra Part C training run, and broader tokenization ablations require additional label construction and retraining beyond the core pipeline.

After running the validation section and the final test section once the final configuration is chosen, the results section should be completed with the saved metrics, confusion matrices, example error cases, ablation tables, and final legal-courtesy verification summary from `results/`.

References

Alex Graves, Santiago Fernandez, Faustino Gomez, and Jurgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, 2006.

Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 1966.

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.